

minimizing the sum $f(S, C)$ of the squared distance from each point to its nearest center, where

$$\begin{aligned} f(S, C) &= \sum_{\mathbf{x} \in S} \min \{ \Delta(\mathbf{x}, \mathbf{c}^{(j)}) : 1 \leq j \leq k \} \\ &= \sum_{\ell=1}^k \sum_{\mathbf{x} \in S^{(\ell)}} \Delta(\mathbf{x}, \mathbf{c}^{(\ell)}) . \end{aligned} \quad (33.2)$$

In the second line, the k -clustering $\langle S^{(1)}, S^{(2)}, \dots, S^{(k)} \rangle$ is defined by the centers C and the nearest-center rule. See Exercise 33.1-1 for an alternative formulation based on pairwise interpoint distances.

Is there a polynomial-time algorithm for the k -means problem? Probably not, because it is NP-hard [310]. As we'll see in Chapter 34, NP-hard problems have no known polynomial-time algorithm, but nobody has ever proven that polynomial-time algorithms for NP-hard problems cannot exist. Although we know of no polynomial-time algorithm that finds the global minimum over all clusterings (according to equation (33.2)), we *can* find a local minimum.

Lloyd [304] proposed a simple procedure that finds a sequence C of k centers that yields a local minimum of $f(S, C)$. A local minimum in the k -means problem satisfies two simple properties: each cluster has an optimal center (defined below), and each point is assigned to the cluster (or one of the clusters) with the closest center. Lloyd's procedure finds a good clustering—possibly optimal—that satisfies these two properties. These properties are necessary, but not sufficient, for optimality.

Optimal center for a given cluster

In an optimal solution to the k -means problem, each center point must be the *centroid*, or *mean*, of the points in its cluster. The centroid is a d -dimensional point, where the value in each dimension is the mean of the values of all the points in the cluster in that dimension (that is, the mean of the corresponding attribute values in the cluster). That is, if $\mathbf{c}^{(\ell)}$ is the centroid for cluster $S^{(\ell)}$, then for attributes $a = 1, 2, \dots, d$, we have

$$c_a^{(\ell)} = \frac{1}{|S^{(\ell)}|} \sum_{\mathbf{x} \in S^{(\ell)}} x_a .$$

Over all attributes, we write

$$\mathbf{c}^{(\ell)} = \frac{1}{|S^{(\ell)}|} \sum_{\mathbf{x} \in S^{(\ell)}} \mathbf{x} . \quad (33.3)$$

Theorem 33.1

Given a nonempty cluster $S^{(\ell)}$, its centroid (or mean) is the unique choice for the cluster center $\mathbf{c}^{(\ell)} \in \mathbb{R}^d$ that minimizes

$$\sum_{\mathbf{x} \in S^{(\ell)}} \Delta(\mathbf{x}, \mathbf{c}^{(\ell)}) .$$

Proof We wish to minimize, by choosing $\mathbf{c}^{(\ell)} \in \mathbb{R}^d$, the sum

$$\begin{aligned} \sum_{\mathbf{x} \in S^{(\ell)}} \Delta(\mathbf{x}, \mathbf{c}^{(\ell)}) &= \sum_{\mathbf{x} \in S^{(\ell)}} \sum_{a=1}^d (x_a - c_a^{(\ell)})^2 \\ &= \sum_{a=1}^d \left(\sum_{\mathbf{x} \in S^{(\ell)}} x_a^2 - 2 \left(\sum_{\mathbf{x} \in S^{(\ell)}} x_a \right) c_a^{(\ell)} + |S^{(\ell)}| (c_a^{(\ell)})^2 \right) . \end{aligned}$$

For each attribute a , the term summed is a convex quadratic function in $c_a^{(\ell)}$. To minimize this function, take its derivative with respect to $c_a^{(\ell)}$ and set it to 0:

$$-2 \sum_{\mathbf{x} \in S^{(\ell)}} x_a + 2 |S^{(\ell)}| c_a^{(\ell)} = 0$$

or, equivalently,

$$c_a^{(\ell)} = \frac{1}{|S^{(\ell)}|} \sum_{\mathbf{x} \in S^{(\ell)}} x_a .$$

Since the minimum is obtained uniquely when each coordinate of $c_a^{(\ell)}$ is the average of the corresponding coordinate for $\mathbf{x} \in S^{(\ell)}$, the overall minimum is obtained when $\mathbf{c}^{(\ell)}$ is the centroid of the points $\mathbf{x}$, as in equation (33.3). ■

Optimal clusters for given centers

The following theorem shows that the nearest-center rule—assigning each point $\mathbf{x}$ to one of the clusters whose center is nearest to $\mathbf{x}$ —yields an optimal solution to the k -means problem.

Theorem 33.2

Given a set S of n points and a sequence $\langle \mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \dots, \mathbf{c}^{(k)} \rangle$ of k centers, a clustering $\langle S^{(1)}, S^{(2)}, \dots, S^{(k)} \rangle$ minimizes

$$\sum_{\ell=1}^k \sum_{\mathbf{x} \in S^{(\ell)}} \Delta(\mathbf{x}, \mathbf{c}^{(\ell)}) \tag{33.4}$$

if and only if it assigns each point $\mathbf{x} \in S$ to a cluster $S^{(\ell)}$ that minimizes $\Delta(\mathbf{x}, \mathbf{c}^{(\ell)})$.

Proof The proof is straightforward: each point $\mathbf{x} \in S$ contributes exactly once to the sum (33.4), and choosing to put $\mathbf{x}$ in a cluster whose center is nearest minimizes the contribution from $\mathbf{x}$. ■

Lloyd's procedure

Lloyd's procedure just iterates two operations—assigning points to clusters based on the nearest-center rule, followed by recomputing the centers of clusters to be their centroids—until the results converge. Here is Lloyd's procedure:

Input: A set S of points in $\mathbb{R}^d$, and a positive integer k .

Output: A k -clustering $\langle S^{(1)}, S^{(2)}, \dots, S^{(k)} \rangle$ of S with a sequence of centers $\langle \mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \dots, \mathbf{c}^{(k)} \rangle$.

1. **Initialize centers:** Generate an initial sequence $\langle \mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \dots, \mathbf{c}^{(k)} \rangle$ of k centers by picking k points independently from S at random. (If the points are not necessarily distinct, see Exercise 33.1-3.) Assign all points to cluster $S^{(1)}$ to begin.
2. **Assign points to clusters:** Use the nearest-center rule to define the clustering $\langle S^{(1)}, S^{(2)}, \dots, S^{(k)} \rangle$. That is, assign each point $\mathbf{x} \in S$ to a cluster $S^{(\ell)}$ having a nearest center (breaking ties arbitrarily, but not changing the assignment for a point $\mathbf{x}$ unless the new cluster center is strictly closer to $\mathbf{x}$ than the old one).
3. **Stop if no change:** If step 2 did not change the assignments of any points to clusters, then stop and return the clustering $\langle S^{(1)}, S^{(2)}, \dots, S^{(k)} \rangle$ and the associated centers $\langle \mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \dots, \mathbf{c}^{(k)} \rangle$. Otherwise, go to step 4.
4. **Recompute centers as centroids:** For $\ell = 1, 2, \dots, k$, compute the center $\mathbf{c}^{(\ell)}$ of cluster $S^{(\ell)}$ as the centroid of the points in $S^{(\ell)}$. (If $S^{(\ell)}$ is empty, let $\mathbf{c}^{(\ell)}$ be the zero vector.) Then go to step 2.

It is possible for some of the clusters returned to be empty, particularly if many of the input points are identical.

Lloyd's procedure always terminates. By Theorem 33.1, recomputing the centers of each cluster as the cluster centroid cannot increase $f(S, C)$. Lloyd's procedure ensures that a point is reassigned to a different cluster only when such an operation strictly decreases $f(S, C)$. Thus each iteration of Lloyd's procedure, except the last iteration, must strictly decrease $f(S, C)$. Since there are only a finite number of possible k -clusterings of S (at most k^n), the procedure must terminate. Furthermore, once one iteration of Lloyd's procedure yields no decrease in f , further iterations would not change anything, and the procedure can stop at this locally optimum assignment of points to clusters.

If Lloyd's procedure really required k^n iterations, it would be impractical. In practice, it sometimes suffices to terminate the procedure when the percentage decrease in $f(S, C)$ in the latest iteration falls below a predetermined threshold. Because Lloyd's procedure is guaranteed to find only a locally optimal clustering, one approach to finding a good clustering is to run Lloyd's procedure many times with different randomly chosen initial centers, taking the best result.

The running time of Lloyd's procedure is proportional to the number T of iterations. In one iteration, assigning points to clusters based on the nearest-center rule requires $O(dkn)$ time, and recomputing new centers for each cluster requires $O(dn)$ time (because each point is in one cluster). The overall running time of the k -means procedure is thus $O(Tdkn)$.

Lloyd's algorithm illustrates an approach common to many machine-learning algorithms:

- First, define a hypothesis space in terms an appropriate sequence θ of parameters, so that each θ is associated with a specific hypothesis h_θ . (For the k -means problem, θ is a dk -dimensional vector, equivalent to C , containing the d -dimensional center of each of the k clusters, and h_θ is the hypothesis that each data point $\mathbf{x}$ should be grouped with a cluster having a center closest to $\mathbf{x}$.)
- Second, define a measure $f(E, \theta)$ describing how poorly hypothesis h_θ fits the given training data E . Smaller values of $f(E, \theta)$ are better, and a (locally) optimal solution (locally) minimizes $f(E, \theta)$. (For the k -means problem, $f(E, \theta)$ is just $f(S, C)$.)
- Third, given a set of training data E , use a suitable optimization procedure to find a value of θ^* that minimizes $f(E, \theta^*)$, at least locally. (For the k -means problem, this value of θ^* is the sequence C of k center points returned by Lloyd's algorithm.)
- Return θ^* as the answer.

In this framework, we see that optimization becomes a powerful tool for machine learning. Using optimization in this way is flexible. For example, *regularization* terms can be incorporated in the function to be minimized, in order to penalize hypotheses that are “too complicated” and that “overfit” the training data. (Regularization is a complex topic that isn't pursued further here.)

Examples

Figure 33.1 demonstrates Lloyd's procedure on a set of $n = 49$ cities: 48 U.S. state capitals and the District of Columbia. Each city has $d = 2$ dimensions: latitude and longitude. The initial clustering in part (a) of the figure has the initial cluster centers arbitrarily chosen as the capitals of Arkansas, Kansas, Louisiana,

and Tennessee. As the procedure iterates, the value of the function f decreases, until the 11th iteration in part (l), where it remains the same as in the 10th iteration in part (k). Lloyd's procedure then terminates with the clusters shown in part (l).

As Figure 33.2 shows, Lloyd's procedure can also apply to "vector quantization." Here, the goal is to reduce the number of distinct colors required to represent a photograph, thereby allowing the photograph to be greatly compressed (albeit in a lossy manner). In part (a) of the figure, an original photograph 700 pixels wide and 500 pixels high uses 24 bits (three bytes) per pixel to encode a triple of red, green, and blue (RGB) primary color intensities. Parts (b)–(e) of the figure show the results of using Lloyd's procedure to compress the picture from a initial space of 2^{24} possible values per pixel to a space of only $k = 4, k = 16, k = 64$, or $k = 256$ possible values per pixel; these k values are the cluster centers. The photograph can then be represented with only 2, 4, 6, or 8 bits per pixel, respectively, instead of the 24-bits per pixel needed by the initial photograph. An auxiliary table, the "palette," accompanies the compressed image; it holds the k 24-bit cluster centers and is used to map each pixel value to its 24-bit cluster center when the photo is decompressed.

Exercises

33.1-1

Show that the objective function $f(S, C)$ of equation (33.2) may be alternatively written as

$$f(S, C) = \sum_{\ell=1}^k \frac{1}{2|S^{(\ell)}|} \sum_{\mathbf{x} \in S^{(\ell)}} \sum_{\mathbf{y} \in S^{(\ell)}: \mathbf{x} \neq \mathbf{y}} \Delta(\mathbf{x}, \mathbf{y}) .$$

33.1-2

Give an example in the plane with $n = 4$ points and $k = 2$ clusters where an iteration of Lloyd's procedure does not improve $f(S, C)$, yet the k -clustering is not optimal.

33.1-3

When the input to Lloyd's procedure contains many repeated points, a different initialization procedure might be used. Describe a way to pick a number of centers at random that maximizes the number of distinct centers picked. (*Hint*: See Exercise 5.3-5.)

33.1-4

Show how to find an optimal k -clustering in polynomial time when there is just one attribute ($d = 1$).



(a) Original

(b) $k = 4$ ($f = 1.29 \times 10^9$; 31 iterations)(c) $k = 16$ ($f = 3.31 \times 10^8$; 36 iterations)(d) $k = 64$ ($f = 5.50 \times 10^7$; 59 iterations)(e) $k = 256$ ($f = 1.52 \times 10^7$; 104 iterations)

Figure 33.2 Using Lloyd’s procedure for vector quantization to compress a photo by using fewer colors. **(a)** The original photo has 350,000 pixels (700×500), each a 24-bit RGB (red/blue/green) triple of 8-bit values; these pixels (colors) are the “points” to be clustered. Points repeat, so there are only 79,083 distinct colors (less than 2^{24}). After compression, only k distinct colors are used, so each pixel is represented by only $\lceil \lg k \rceil$ bits instead of 24. A “palette” maps these values back to 24-bit RGB values (the cluster centers). **(b)–(e)** The same photo with $k = 4, 16, 64$, and 256 colors. (Photo from standuppaddle, pixabay.com.)

33.2 Multiplicative-weights algorithms

This section considers problems that require you to make a series of decisions. After each decision you receive feedback as to whether your decision was correct. We will study a class of algorithms that are called *multiplicative-weights algorithms*. This class of algorithms has a wide variety of applications, including game playing in economics, approximately solving linear-programming and multicommodity-flow problems, and various applications in online machine learning. We emphasize the online nature of the problem here: you have to make a sequence of decisions, but some of the information needed to make the i th decision appears only after you have already made the $(i - 1)$ st decision. In this section, we look at one particular problem, known as “learning from experts,” and develop an example of a multiplicative-weights algorithm, called the weighted-majority algorithm.

Suppose that a series of events will occur, and you want to make predictions about these events. For example, over a series of days, you want to predict whether it is going to rain. Or perhaps you want to predict whether the price of a stock will increase or decrease. One way to approach this problem is to assemble a group of “experts” and use their collective wisdom in order to make good predictions. Let’s denote the experts, n of them, by $E_1, E_2, \dots, E_n$, and let’s say that T events are going to take place. Each event has an outcome of either 0 or 1, with $o^{(t)}$ denoting the outcome of the t th event. Before event t , each expert $E^{(i)}$ makes a prediction $q_i^{(t)} \in \{0, 1\}$. You, as the “learner,” then take the set of n expert predictions for event t and produce a single prediction $p^{(t)} \in \{0, 1\}$ of your own. You base your prediction only on the predictions of the experts and anything you have learned about the experts from their previous predictions. You do not use any additional information about the event. Only after making your prediction do you ascertain the outcome $o^{(t)}$ of event t . If your prediction $p^{(t)}$ matches $o^{(t)}$, then you were correct; otherwise, you made a mistake. The goal is to minimize the total number m of mistakes, where $m = \sum_{t=1}^T |p^{(t)} - o^{(t)}|$. You can also keep track of the number of mistakes each expert makes: expert E_i makes m_i mistakes, where $m_i = \sum_{t=1}^T |q_i^{(t)} - o^{(t)}|$.

For example, suppose that you are following the price of a stock, and each day you decide whether to invest in it for just that day by buying it at the beginning of the day and selling it at the end of the day. If, on some day, you buy the stock and it goes up, then you made the correct decision, but if the stock goes down, then you made a mistake. Similarly, if on some day, you do not buy the stock and it goes down, then you made the correct decision, but if the stock goes up, then you made a mistake. Since you would like to make as few mistakes as possible, you use the advice of the experts to make your decisions.

We'll assume nothing about the movement of the stock. We'll also assume nothing about the experts: the experts' predictions could be correlated, they could be chosen to deceive you, or perhaps some are not really experts after all. What algorithm would you use?

Before designing an algorithm for this problem, we need to consider what is a fair way to evaluate our algorithm. It is reasonable to expect that our algorithm performs better when the expert predictions are better, and that it performs worse when the expert predictions are worse. The goal of the algorithm is to limit the number of mistakes you make to be close to the number of mistakes that the best of the experts makes. At first, this goal might seem impossible, because you do not know until the end which expert is best. We'll see, however, that by taking the advice provided by all the experts into account, you can achieve this goal. More formally, we use the notion of "regret," which compares our algorithm to the performance of the best expert (in hindsight) over all. Letting $m^* = \min \{m_i : 1 \leq i \leq n\}$ denote the number of mistakes made by the best expert, the *regret* is $m - m^*$. The goal is to design an algorithm with low regret. (Regret can be negative, although it typically isn't, since it is rare that you do better than the best expert.)

As a warm-up, let's consider the case in which one of the experts makes a correct prediction each time. Even without knowing who that expert is, you can still achieve good results.

Lemma 33.3

Suppose that out of n experts, there is one who always makes the correct prediction for all T events. Then there is an algorithm that makes at most $\lceil \lg n \rceil$ mistakes.

Proof The algorithm maintains a set S consisting of experts who have not yet made a mistake. Initially, S contains all n experts. The algorithm's prediction is always the majority vote of the predictions of the experts remaining in set S . In case of a tie, the algorithm makes any prediction. After each outcome is learned, set S is updated to remove all the experts who made an incorrect prediction about that outcome.

We now analyze the algorithm. The expert who always makes the correct prediction will always be in set S . Every time the algorithm makes a mistake, at least half of the experts who were still in S also make a mistake, and these experts are removed from S . If S' is the set of experts remaining after removing those who made a mistake, we have that $|S'| \leq |S|/2$. The size of S can be halved at most $\lceil \lg n \rceil$ times until $|S| = 1$. From this point on, we know that the algorithm never makes a mistake, since the set S consists only of the one expert who never makes a mistake. Therefore, overall the algorithm makes at most $\lceil \lg n \rceil$ mistakes. ■

Exercise 33.2-1 asks you to generalize this result to the case when there is no expert who makes perfect predictions and show that, for any set of experts, there is an algorithm that makes at most $m^* \lceil \lg n \rceil$ mistakes. The generalized algorithm begins in the same way. The set S might become empty at some point, however. If that ever happens, reset S to contain all the experts and continue the algorithm.

You can substantially improve your prediction ability by not just tracking which experts have not made any mistakes, or have not made any mistakes recently, to a more nuanced evaluation of the quality of each expert. The key idea is to use the feedback you receive to update your evaluation of how much trust to put in each expert. As the experts make predictions, you observe whether they were correct and decrease your confidence in the experts who make more mistakes. In this way, you can learn over time which experts are more reliable and which are less reliable, and weight their predictions accordingly. The change in weights is accomplished via multiplication, hence the term “multiplicative weights.”

The algorithm appears in the procedure WEIGHTED-MAJORITY on the following page, which takes a set $E = \{E_1, E_2, \dots, E_n\}$ of experts, a number T of events, the number n of experts, and a parameter $0 < \gamma \leq 1/2$ that controls how the weights change. The algorithm maintains weights $w_i^{(t)}$ for $i = 1, 2, \dots, n$ and $t = 1, 2, \dots, T$, where $0 < w_i^{(t)} \leq 1$. The **for** loop of lines 1–2 sets the initial weights $w_i^{(1)}$ to 1, capturing the idea that with no knowledge, you trust each expert equally. Each iteration of the main **for** loop of lines 3–18 does the following for an event $t = 1, 2, \dots, T$. Each expert E_i makes a prediction for event t in line 4. Lines 5–8 compute $upweight^{(t)}$, the sum of the weights of the experts who predict 1 for event t , and $downweight^{(t)}$, the sum of the weights of the experts who predict 0 for the event. Lines 9–11 decide the algorithm’s prediction $p^{(t)}$ for event t based on whichever weighted sum is larger (breaking ties in favor of deciding 1). The outcome of event t is revealed in line 12. Finally, lines 14–17 decrease the weights of the experts who made an incorrect prediction for event t by multiplying their weights by $1 - \gamma$, leaving alone the weights of the experts who correctly predicted the event’s outcome. Thus, the fewer mistakes each expert makes, the higher that expert’s weight.

The WEIGHTED-MAJORITY procedure doesn’t do much worse than any expert. In particular, it doesn’t do much worse than the best expert. To quantify this claim, let $m^{(t)}$ be the number of mistakes made by the procedure through event t , and let $m_i^{(t)}$ be the number of mistakes made by expert E_i through event t . The following theorem is the key.

```

WEIGHTED-MAJORITY( $E, T, n, \gamma$ )
1  for  $i = 1$  to  $n$ 
2       $w_i^{(1)} = 1$                                 // trust each expert equally
3  for  $t = 1$  to  $T$ 
4      each expert  $E_i \in E$  makes a prediction  $q_i^{(t)}$ 
5       $U = \{E_i : q_i^{(t)} = 1\}$                     // experts who predicted 1
6       $upweight^{(t)} = \sum_{i:E_i \in U} w_i^{(t)}$     // sum of weights of who predicted 1
7       $D = \{E_i : q_i^{(t)} = 0\}$                     // experts who predicted 0
8       $downweight^{(t)} = \sum_{i:E_i \in D} w_i^{(t)}$  // sum of weights of who predicted 0
9      if  $upweight^{(t)} \geq downweight^{(t)}$ 
10          $p^{(t)} = 1$                                 // algorithm predicts 1
11     else  $p^{(t)} = 0$                                 // algorithm predicts 0
12     outcome  $o^{(t)}$  is revealed
13     // If  $p^{(t)} \neq o^{(t)}$ , the algorithm made a mistake.
14     for  $i = 1$  to  $n$ 
15         if  $q_i^{(t)} \neq o^{(t)}$                     // if expert  $E^{(i)}$  made a mistake ...
16              $w_i^{(t+1)} = (1 - \gamma)w_i^{(t)}$  // ... then decrease that expert's weight
17         else  $w_i^{(t+1)} = w_i^{(t)}$ 
18     return  $p^{(t)}$ 

```

Theorem 33.4

When running WEIGHTED-MAJORITY, we have, for every expert E_i and every event $T' \leq T$,

$$m^{(T')} \leq 2(1 + \gamma)m_i^{(T')} + \frac{2 \ln n}{\gamma}. \quad (33.5)$$

Proof Every time an expert E_i makes a mistake, its weight, which is initially 1, is multiplied by $1 - \gamma$, and so we have

$$w_i^{(t)} = (1 - \gamma)^{m_i^{(t)}} \quad (33.6)$$

for $t = 1, 2, \dots, T$.

We use a potential function $W(t) = \sum_{i=1}^n w_i^{(t)}$, summing the weights for all n experts after iteration t of the **for** loop of lines 3–18. Initially, we have $W(0) = n$ since all n weights start out with the value 1. Because each expert belongs to either the set U or the set D (defined in lines 5 and 7 of WEIGHTED-MAJORITY), we always have $W(t) = upweight^{(t)} + downweight^{(t)}$ after each execution of line 8.

Consider an iteration t in which the algorithm makes a mistake in its prediction, which means that either the algorithm predicts 1 and the outcome is 0 or the al-